

CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

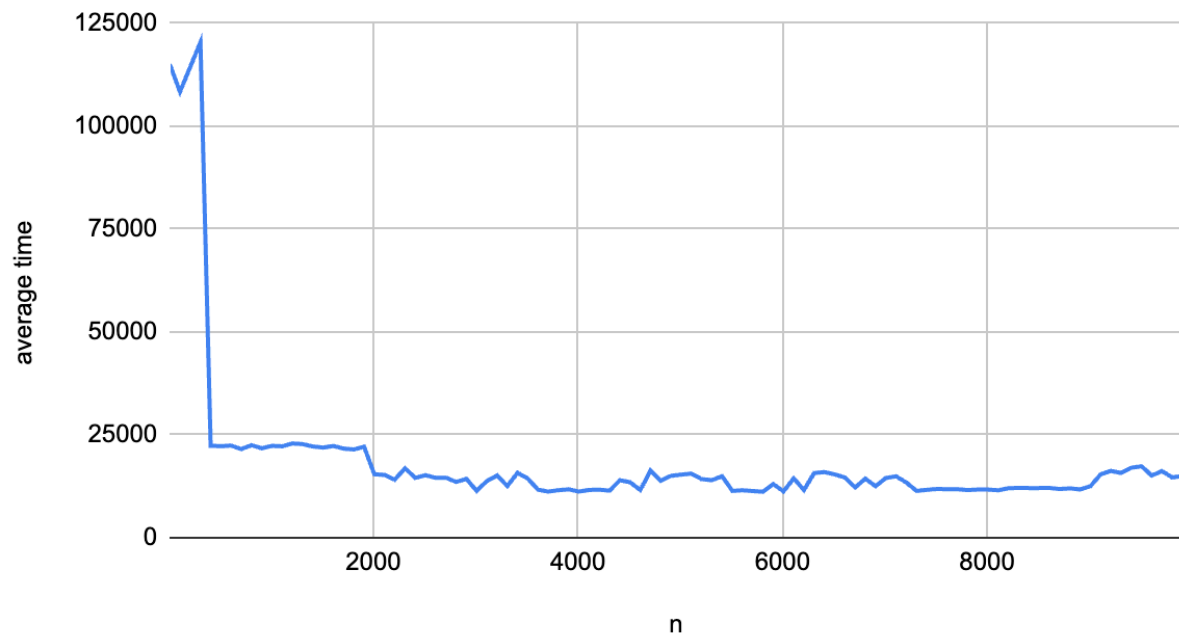
Exercise 0 Benchmarking Worksheet (26wi)

Question 1:

For all of the future questions we will need to compare the benchmarking results for the 6 classes ArrayQueue, LinkedQueue, ArrayStack, LinkedStack, NaiveQueue, and NaiveStack. Include your charts in the corresponding box below:

ArrayQueue

average time vs. n

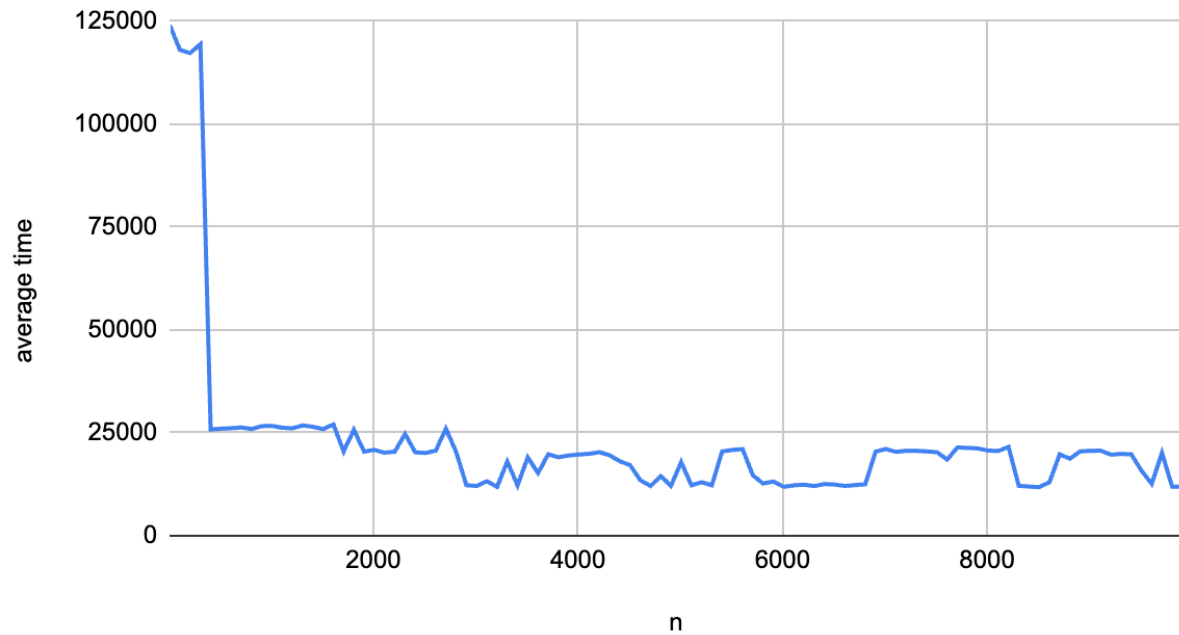


CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

LinkedList

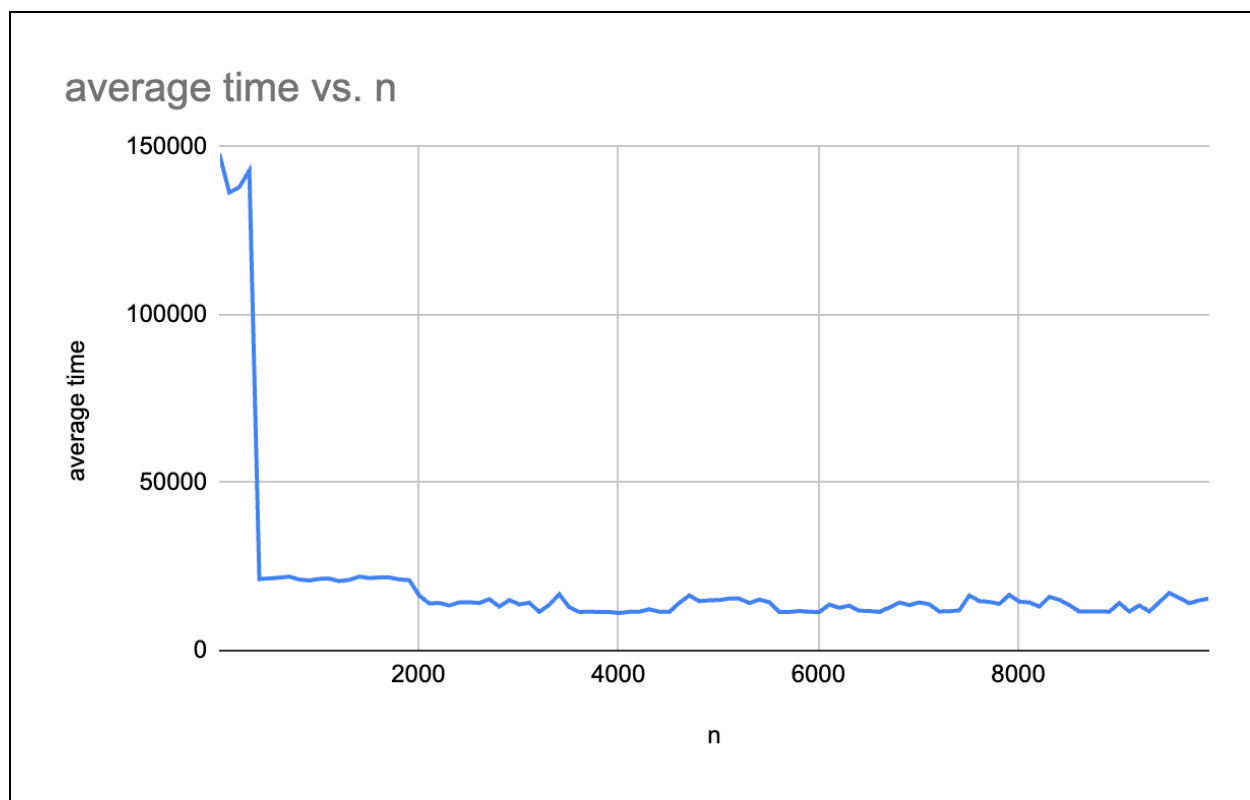
average time vs. n



ArrayStack

CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

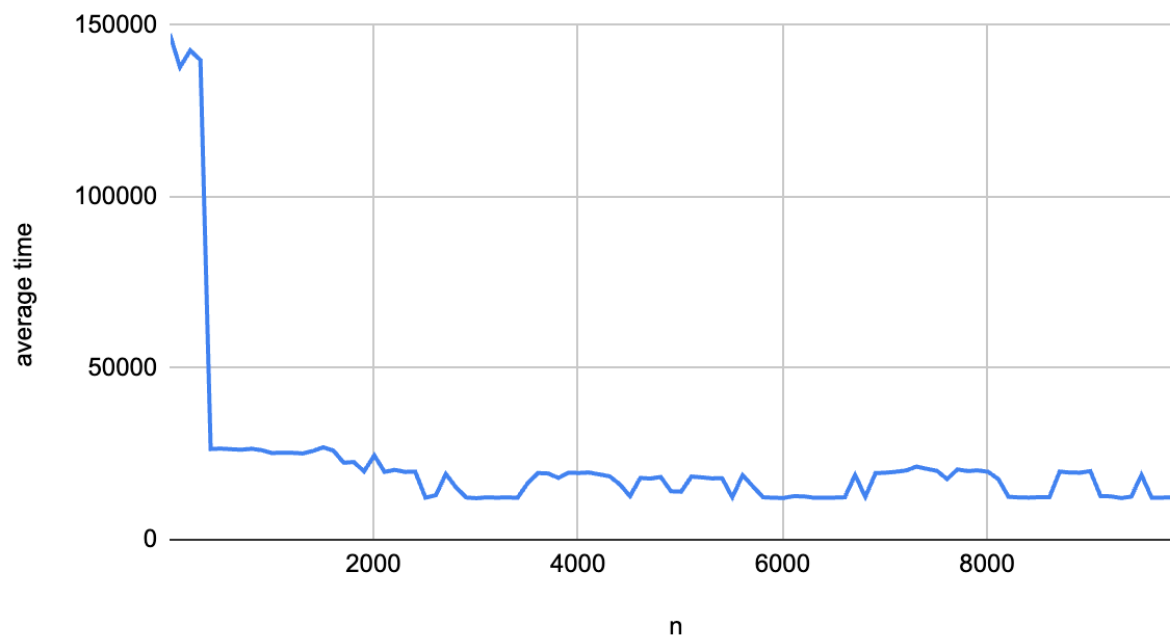


CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

LinkedStack

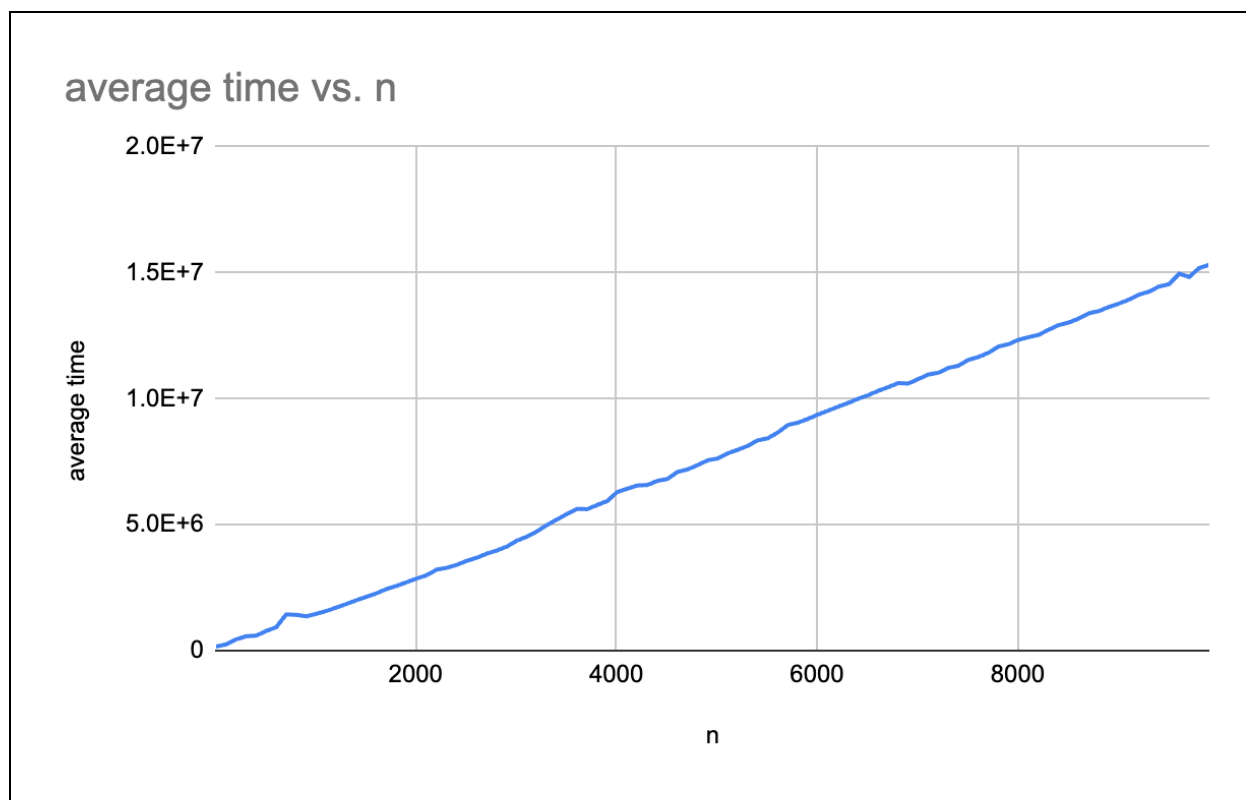
average time vs. n



NaiveQueue

CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

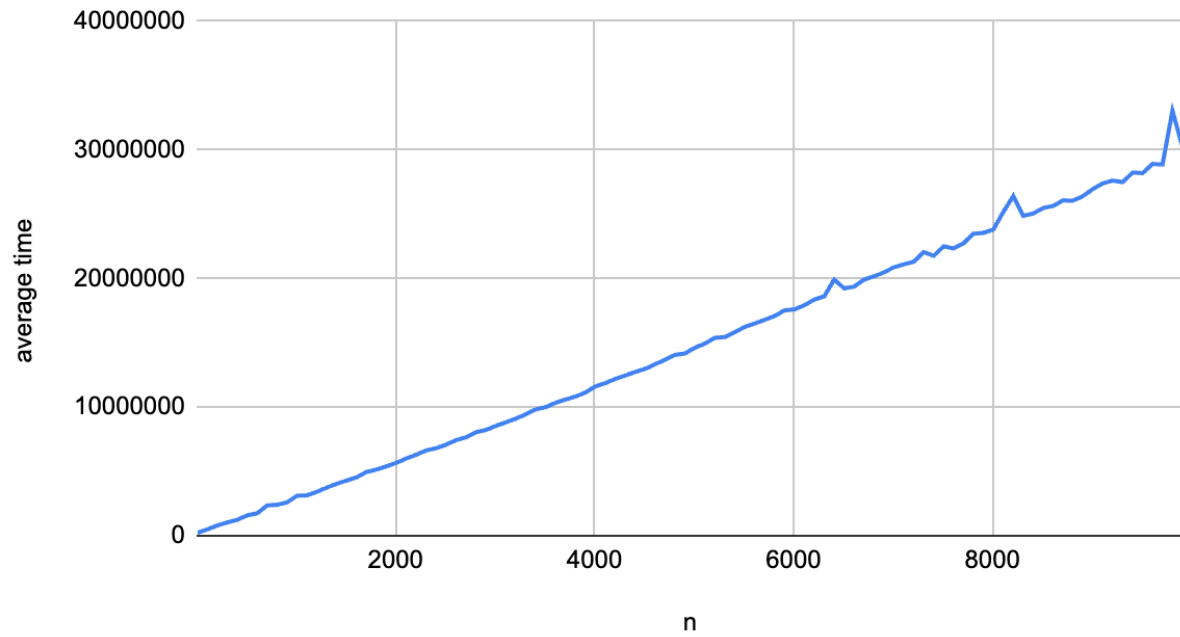


CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

NaiveStack

average time vs. n



CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet

Question 2:

If all went according to plan, all four of the data structures that you implemented should run in constant time. This should be evident in your charts above by noting that the running time trends to a horizontal line. By comparison, the running times of NaiveQueue and NaiveStack increase as n gets larger, meaning that the running time is worse than constant. In fact, the running time of adding plus removing is linear, i.e. $O(n)$. For each of these classes at least one of enqueue/dequeue or push/pop is linear time. For this question, reference the implementations of these classes, identify which operation is linear time (or else say both are), and justify why it runs in linear time (1-2 sentences should be sufficient).

NaiveQueue - Which operation is linear time? (enqueue, dequeue, or both)

Both

Justification:

Both enqueue and dequeue are in native time because both requires shifting all elements in the structure, so its all $O(n)$ time.

NaiveStack - Which operation is linear time? (push, pop, or both)

Both

Justification:

Similar to a NaiveQueue, both push and pop is implemented by inserting/removing elements, and thus shifting all elements, which creates linear time.

Question 3:

For this question, we will look at why it's important to double the size of the array when resizing in the array-based data structures.

First, modify your ArrayStack data structure so that instead of doubling the size of your array at resizing, you add 5 to the size of the array. After making this change, run the benchmark again for ArrayStack. At this point, you should notice that the benchmark takes SUBSTANTIALLY longer (run your code long enough to observe this, but it will be painful if you wait for it to finish). Summarize why you believe the running time was so much worse! (1-2 sentences should suffice)

Adding a constant amount instead of doubling forces resizing to happen much more often ($O(n)$ times more frequently for n elements). This results in a much larger time complexity because this is quadratic ($O(n^2)$) rather than doubling having the resizing happening only $O(\log_n)$ times.

CSE 332: Data Structures and Parallelism

Exercise 0 - Benchmarking Worksheet